

# Ausgewählte Datenstrukturen in Java

## ArrayList vs. HashMap

| Eigenschaft                     | ArrayList                                                                                                              | HashMap                                                                                                                     |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Datenstrukturtyp                | Dynamisches Array, implementiert das List-Interface                                                                    | Hash-basierte assoziative Datenstruktur, implementiert das Map-Interface                                                    |
| Kernfunktionalität              | Speicherung geordneter Elemente mit indexbasiertem Zugriff                                                             | Speicherung von Schlüssel-Wert-Paaren für schnellen, direkten Zugriff über den Schlüssel                                    |
| Interne Datenstruktur           | Internes Array, das bei Bedarf dynamisch vergrößert wird                                                               | Array von Buckets; jeder Bucket enthält bei hohen Kollisionsraten entweder verkettete Listen oder ab Java 8 auch Binärbäume |
| Zugriffsordnung                 | Elemente behalten ihre Einfügereihenfolge; direkter Zugriff über den Index                                             | Keine garantierte Reihenfolge (außer bei Verwendung von LinkedHashMap, welche die Einfügereihenfolge erhält)                |
| Zeitkomplexität (Operationen)   | Zugriff: $O(1)$ Hinzufügen am Ende: Amortisiert $O(1)$ Einfügen/Entfernen an beliebiger Position: $O(n)$               | put/get: Durchschnittlich $O(1)$ , im Worst-Case $O(n)$ (bei vielen Kollisionen)                                            |
| Synchronisation / Thread-Safety | Nicht synchronisiert; bei paralleler Nutzung externe Synchronisation erforderlich (z. B. Collections.synchronizedList) | Nicht synchronisiert; für thread-sichere Operationen: z. B. ConcurrentHashMap oder explizite Synchronisation                |
| Null-Werte Unterstützung        | Erlaubt null-Werte                                                                                                     | Erlaubt einen null-Schlüssel (nur eine) sowie mehrere null-Werte als Werte                                                  |
| Iterierbarkeit                  | Unterstützt Iteratoren und ListIteratoren; bietet sequentiellen und indexbasierten Zugriff                             | Iteration über keySet, values oder entrySet; keine feste Reihenfolge, da diese von der internen Hash-Verteilung abhängt     |

| Eigenschaft          | ArrayList                                                                                                                                       | HashMap                                                                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Verwendungsszenarien | Geeignet für dynamische, geordnete Sammlungen, bei denen schneller, direkter Indexzugriff benötigt wird; häufig in Listenoperationen eingesetzt | Ideal für schnellen Zugriff auf Werte anhand von Schlüsseln, z. B. in Caching-Szenarien, Lookup-Tabellen oder zum Implementieren von Zuordnungen und Mappings |

## Queue vs. Stack

| Eigenschaft                     | Queue                                                                                                                                                  | Stack                                                                                                                                                 |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Datenstrukturtyp                | Schnittstelle für Warteschlangen (FIFO-Prinzip), häufig implementiert als LinkedList oder ringpufferbasiertes Array                                    | Klassische LIFO-Datenstruktur; historisch als Erweiterung von Vector implementiert, heutzutage oft durch Deque-Implementierungen ersetzt              |
| Kernfunktionalität              | Verwaltung von Elementen in der Reihenfolge ihres Eingangs („first in, first out“)                                                                     | Verwaltung von Elementen nach dem Last-In-First-Out-Prinzip; zuletzt hinzugefügtes Element wird zuerst entfernt                                       |
| Interne Datenstruktur           | Abhängig von der konkreten Implementierung: Verkettete Listen (z. B. LinkedList) Array-basierte Implementierungen (bei Circular Queues)                | Basierend auf einem dynamischen Array (bei klassischen Implementierungen wie Stack, der von Vector erbt) oder moderner über Deque realisiert          |
| Zugriffsordnung                 | FIFO: Elemente werden in der Reihenfolge verarbeitet, in der sie hinzugefügt wurden                                                                    | LIFO: Das zuletzt hinzugefügte Element ist das erste, das entfernt oder abgefragt wird                                                                |
| Zeitkomplexität (Operationen)   | Enqueue (Anfügen) und Dequeue (Entfernen): Idealerweise $O(1)$ Suche (bei sequentiell durchlaufen): $O(n)$                                             | push/pop/peek: Typischerweise $O(1)$ ; direkte Operationen auf dem oberen Ende des Stacks                                                             |
| Synchronisation / Thread-Safety | Abhängig von der Implementierung: Standardimplementierungen (z. B. LinkedList) sind meist nicht synchronisiert Alternativen: ConcurrentLinkedList etc. | Der klassische Stack (basierend auf Vector) ist synchronisiert; moderne Alternativen (z. B. Deque) bieten bessere Performance bei Mehrthreadszenarien |

| Eigenschaft                     | Queue                                                                                                                                                                | Stack                                                                                                                                                                        |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Null-Werte Unterstützung</b> | Häufig nicht erlaubt (z. B. in PriorityQueue) oder abhängig von der konkreten Implementierung; oft wird null als spezielles Signal interpretiert                     | Unterstützt null-Werte, da die Basis (Vector oder Deque) dies zulässt                                                                                                        |
| <b>Iterierbarkeit</b>           | Iterator liefert in der Regel die Elemente in der FIFO-Reihenfolge, sofern die konkrete Implementierung diese beibehält                                              | Bietet Iteratoren; jedoch entspricht die Iterationsreihenfolge nicht zwingend der LIFO-Struktur – klassische LIFO-Operationen erfolgen über push/pop                         |
| <b>Verwendungsszenarien</b>     | Geeignet für Aufgaben oder Event-Management, bei denen die Reihenfolge des Eingangs wichtig ist, z. B. in Task-Schedulern, Print-Queues oder Event-Dispatch-Systemen | Verwendet in Szenarien, die Rückverfolgung oder Undo-Operationen erfordern, z. B. bei rekursiven Algorithmen, Backtracking, Syntax-Parsing oder zur Verwaltung von Zuständen |